

FloodEngine – Architecture & Function Reference

Internal Technical Documentation

FloodSim GIS Project | Version 1.0 | 2025

0 Contents

1	System Overview	3
1.1	Component Summary	3
1.2	High-Level Data Flow	3
2	FloodEngine API Layer	4
2.1	FloodEngine Struct	4
2.2	Exported Functions	4
2.3	Qt Call Sequence	4
3	Raster Class	5
3.1	Internal Data	5
3.2	Data Loading	5
3.3	Basic Statistics	5
3.4	Flow Direction (D8 Algorithm)	5
3.5	Flow Accumulation	6
3.6	Flood Depth Computation	7
3.7	Flood Statistics	7
3.8	Flow Path Tracing	7
3.9	River Detection	7
4	RandomForest Class	8
4.1	Node Structure	8
4.2	Functions	8
4.3	Feature Vector	8
4.4	Model File Format (rf_model.txt)	8
5	Python Training Pipeline	9
5.1	Workflow	9
5.2	Dataset Schema	9
6	Known Issues & Improvement Plan	10
6.1	Current Issues	10
6.2	Phase 1 Fix Plan	10
6.3	Phase 2 – Qt Time Series (No Engine Changes Needed)	10
7	Full Architecture Diagram	11

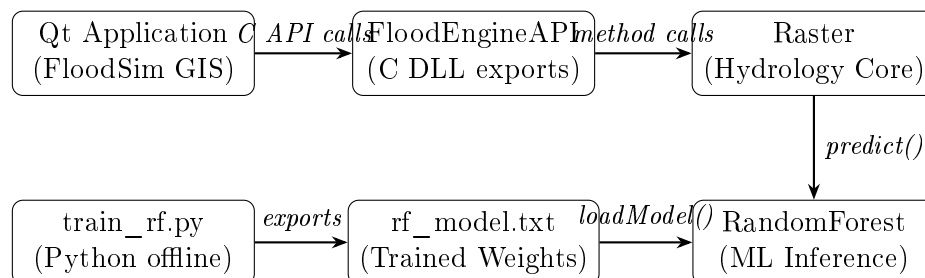
1 System Overview

The FloodEngine is a C++ dynamic library (DLL) providing flood depth prediction for FloodSim GIS. It combines physics-based hydrology (D8 flow accumulation) with a pre-trained Random Forest ML model to produce flood depth grids.

1.1 Component Summary

Component	Language	Role
FloodEngineAPI	C++ DLL	Public C API exported to Qt
Raster	C++	Core grid class – hydrology + flood
RandomForest	C++	Inference for pre-trained RF model
train_rf.py	Python	Offline training, exports rf_model.txt

1.2 High-Level Data Flow



2 FloodEngine API Layer

2.1 FloodEngine Struct

```

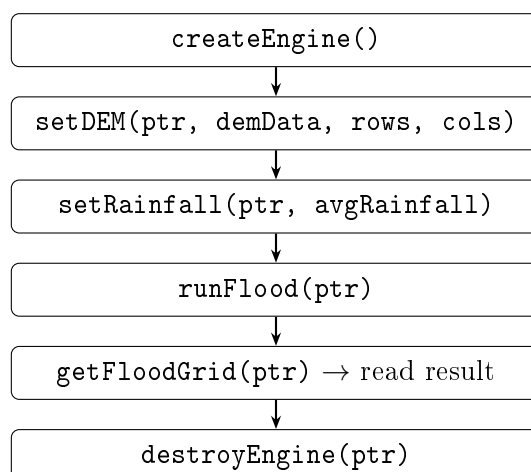
1 struct FloodEngine {
2     Raster raster;           // Core hydrology grid
3     std::vector<double> flood; // Flattened 1D result grid
4     int rows = 0;
5     int cols = 0;
6     double rainfall = 0.0;
7 };

```

2.2 Exported Functions

Function	Signature	Description
createEngine()	void*()	Allocates new FloodEngine on heap. Returns opaque pointer.
destroyEngine()	void(void*)	Frees engine pointer. Call on shutdown.
setDEM()	void(void*,double*,int,int)	Receives flat 1D DEM array from Qt. Reshapes to 2D and calls <code>raster.loadFromGrid()</code> .
setRainfall()	void(void*,double)	Stores rainfall (mm) for use in <code>runFlood()</code> .
runFlood()	void(void*)	Calls <code>FloodDepthGridWithRain(0,rainfall)</code> . Flattens result into <code>engine->flood</code> .
getFloodGrid()	const double*(void*)	Returns raw pointer to flattened flood depth array.
getRows()	int(void*)	Returns row count.
getCols()	int(void*)	Returns column count.

2.3 Qt Call Sequence



3 Raster Class

Raster is the core class storing the DEM grid and implementing all hydrological computations.

3.1 Internal Data

Field	Type	Description
grid	vector<vector<double>>	Elevation grid
rows	int	Number of rows
cols	int	Number of columns

3.2 Data Loading

Function	Description
loadCSVFile(filename)	Reads DEM from CSV line by line. Splits by comma. Sets rows and cols .
loadFromGrid(g)	Direct assignment from vector<vector<double>> . Used by DLL setDEM() .
gridReturn()	Returns a copy of internal grid .

3.3 Basic Statistics

Function	Description
getMax()	Scans entire grid, returns maximum elevation.
getMin()	Scans entire grid, returns minimum elevation.
getAverage()	Sum divided by rows * cols .
getRows()	Returns rows .
getCols()	Returns cols .

3.4 Flow Direction (D8 Algorithm)

D8 Flow Direction

For each cell (i, j) , find the steepest downhill neighbor among all 8 adjacent cells. Encode as integer 0–7 (compass direction). If no lower neighbor exists, return -1 (local minimum / pour point).

Function	Description
getFlowDirection(i, j)	Scans 8 neighbors. Returns direction index 0–7 of steepest descent. Returns -1 if local minimum.
nextCell(i, j, dir)	Converts direction 0–7 to neighbor coordinates using offset arrays. Returns $(-1, -1)$ if out of bounds.
getNeighbours(i, j)	Returns all valid 8-connected neighbors as vector<pair<int, int>> .

Direction Encoding:

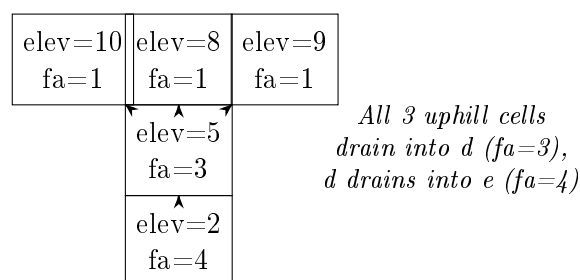
E	SE	S	SW	W	NW	N	NE
0	1	2	3	4	5	6	7

NW=5	N=6	NE=7
W=4	(i,j)	E=0
SW=3	S=2	SE=1

3.5 Flow Accumulation**Recursive Flow Accumulation**

`fa[i][j]` = number of upstream cells draining into (i, j) plus 1. Uses depth-first recursion with `visited[]` grid for memoization. A cell drains into (i, j) if its `nextCell()` returns (i, j) .

Function	Description
<code>computeFlowAccumulation()</code>	Initializes <code>fa[rows][cols]=0</code> and <code>visited[]=false</code> . Calls <code>flowAccumulator(i, j)</code> for every cell. Returns completed <code>fa</code> grid.
<code>flowAccumulator(i, j, fa, visited)</code>	Recursive. Marks cell visited. Sums 1 plus contributions from all neighbors whose flow points to (i, j) . Memoizes in <code>fa[i][j]</code> .

Visual Example:

3.6 Flood Depth Computation

Function	Description
<code>FloodDepthGrid(t)</code>	Gets rainfall via <code>rainTime(t)</code> (hardcoded array). Computes <code>depth[i][j] = rainfall * flowAcc[i][j]</code> .
<code>FloodDepthGridWithRain(t, rainfall)</code>	Same formula but uses provided <code>rainfall</code> directly. This is what the DLL calls. <code>t</code> is currently unused.
<code>MLFloodDepthGrid(t, rf)</code>	Builds feature vector per cell and calls <code>rf.predict()</code> . Returns ML-corrected depth grid.
<code>OutflowInflow(t)</code>	Simple routing: <code>outflow = depth * 0.3</code> , routed to downstream cell.

Core Formula:

$$\text{depth}[i][j] = \text{rainfall} \times \text{flowAccumulation}[i][j] \quad (1)$$

Cells with higher upstream drainage area accumulate more water – valleys flood deeper than ridges.

3.7 Flood Statistics

Function	Description
<code>FloodMax(t)</code>	Max value in <code>FloodDepthGrid(t)</code> .
<code>FloodMin(t)</code>	Min value in <code>FloodDepthGrid(t)</code> .
<code>FloodAvg(t)</code>	Sum / (rows * cols).
<code>FloodCells(t)</code>	Count of cells where <code>depth > 0</code> .
<code>FloodVolume(t)</code>	Sum of all depth values.
<code>exportFloodStat(filename, t)</code>	Exports CSV row: time, max, avg, cells, volume.

3.8 Flow Path Tracing

Function	Description
<code>traceFlowPath(i, j, visited)</code>	Recursive. Follows <code>getFlowDirection</code> from start cell until boundary or visited cell. Returns ordered path.
<code>exportFlowPathCSV(file, si, sj)</code>	Traces path from (si, sj) . Exports as CSV: x, y, order.

3.9 River Detection

Function	Description
<code>RiverGrid()</code>	Marks cells as river (1) if <code>flowAccumulation >= 3</code> . Returns binary grid.

4 RandomForest Class

Pure C++ inference engine. Model trained in Python, loaded from text file.

4.1 Node Structure

```

1 struct TreeNode {
2     bool    isLeaf;
3     int     feature;      // 0=rain_t, 1=rain_t1, 2=flow_acc, 3=
        elevation
4     double threshold;
5     int     left;        // left child index
6     int     right;       // right child index
7     double value;        // leaf prediction
8 };

```

4.2 Functions

Function	Description
loadModel(path)	Reads tree count, then each tree: node count, then each node as L value (leaf) or N feat thresh left right.
predict(features)	Traverses each tree from root. At each node: go left if features[f] <= threshold. Average all tree leaf values.

4.3 Feature Vector

Index	Feature	Description
0	rain_t	Rainfall at current time step (mm)
1	rain_t_1	Rainfall at previous time step (mm)
2	flow_acc	Flow accumulation at cell (i, j)
3	elevation	DEM elevation at cell (metres)

4.4 Model File Format (rf_model.txt)

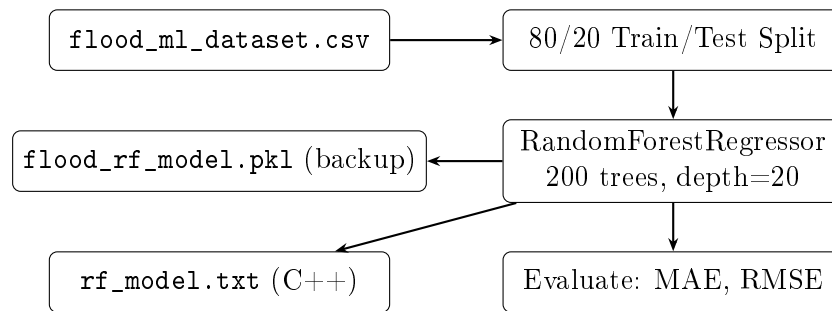
```

200          <- number of trees
45           <- node count for tree 1
N 2 15.5 1 2 <- internal: feature=2, thresh=15.5, left=1, right
    =2
L 3.14       <- leaf: prediction = 3.14
...          <- remaining nodes and trees

```

5 Python Training Pipeline

5.1 Workflow



5.2 Dataset Schema

Column	Name	Description
1–3	cell_id, i, j	Cell identifier and grid position
4	t	Time step index
5	rain_t	Rainfall at time t
6	rain_t_1	Rainfall at $t - 1$
7	flow_acc	Flow accumulation
8	elevation	DEM value
9	depth_t1	Target: physics flood depth at $t + 1$

6 Known Issues & Improvement Plan

6.1 Current Issues

#	Issue	Detail
1	Fake time parameter	<code>t</code> in <code>FloodDepthGridWithRain(t, r)</code> is unused. DLL passes <code>t=0</code> always.
2	Flow accumulation recomputed	Called every time <code>FloodDepthGridWithRain</code> runs. Should be cached after <code>loadFromGrid()</code> .
3	Boundary bug in <code>OutflowInflow</code>	<code>nextCell()</code> can return <code>(-1, -1)</code> for border cells. No bounds check before indexing <code>flood[p.first][p.second]</code> .
4	<code>FloodPercentage()</code> stub	Declared in header, body missing.
5	<code>ValidCell()</code> stub	Declared in header, body missing.

6.2 Phase 1 Fix Plan

#	Task	Action
1	Remove fake <code>t</code>	Create <code>FloodDepthGridFinal(double rainfall)</code> – no <code>t</code> .
2	Cache flow accumulation	Compute once in <code>loadFromGrid()</code> , store as member variable.
3	Fix boundary bug	Add <code>if (p.first < 0) continue;</code> in <code>OutflowInflow</code> .
4	Simplify DLL	Call <code>FloodDepthGridFinal(rainfall)</code> in <code>runFlood()</code> .

6.3 Phase 2 – Qt Time Series (No Engine Changes Needed)

Engine stays simple – one final grid. Qt generates time steps:

$$\text{grid_t}[i][j] = \text{finalGrid}[i][j] \times \text{factor}[t] \quad (2)$$

Hour 1	Hour 2	Hour 3	Hour 4	Hour 5	Hour 6
10%	25%	45%	65%	80%	100%

7 Full Architecture Diagram

